

CPSC 304 Project Cover Page

Milestone #: 4

Date: 04/02/26

Group Number: 36

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Julian Wright	84895358	w6i9k	jwrigh13@student.ubc.ca
Dorson Tang	94262474	d5v6x	dtang08@student.ubc.ca
Krishna Garcha	40548323	g8a8b	garchakri@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

SQL Script

The SQL that drops existing tables and then creates and populates all of the relations is in `create_data.sql`. It also provides the full database schema and clearly lists all keys (primary and foreign) and constraints.

Final Project Description

Our final project is an electronics project management. It allows users to create projects and add electronic components (such as resistors, capacitors, etc.) for organizational/ tracking purposes. They can also choose suppliers for the components and track orders from those suppliers. Users can add locations (e.g., drawers, component bins, etc.) to both their components and projects to be able to find them better, something especially important for the components as they can get lost easily. We were able to achieve a cohesive application that can track all necessary information about components, and it has a very appealing UI for non technical users.

Implemented Queries

Projection

- Found in [location.ts](#):38 in the backend, and `locations.tsx`:168 in the frontend

DELETE

- Component:
 - Backend: [components.ts](#):214
 - Frontend: `ComponentsTable.tsx`:312, `CapacitorTable.tsx`:33, `ResistorTable.tsx`:33, `DiodeTable.tsx`:33
- Supplier:
 - Backend: [shipping.ts](#):74
 - Frontend: `SuppliersTable.tsx`:124
- Courier:
 - Backend: [shipping.ts](#):151
 - Frontend: `CouriersTable.tsx`:124
- Purchase:
 - Backend: [shipping.ts](#):265
 - Frontend: `PurchasesTable.tsx`:183

Join

- Found in [location.ts:35,38](#) for the backend join and WHERE, and locations.tsx:121 for the frontend interface

INSERT

- Suppliers:
 - Backend: [shipping.ts](#)23
 - Frontend: SuppliersTable.tsx:99
- Couriers:
 - Backend: [shipping.ts](#):101
 - Frontend: CouriersTable.tsx:97
- Purchases:
 - Backend: [shipping.ts](#):203
 - Frontend: PurchasesTable.tsx:155

UPDATE:

- Suppliers:
 - Backend: [shipping.ts](#):55
 - Frontend: SuppliersTable.tsx:105
- Couriers:
 - Backend: [shipping.ts](#):134
 - Frontend: CouriersTable.tsx:104
- Purchases:
 - Backend: [shipping.ts](#):246
 - Frontend: PurchasesTable.tsx:164

Update

- Locations:
 - Backend: [location.ts:91](#) and [location.ts:123](#) for creating and renaming locations
 - Frontend
 - Renaming: Locations.tsx:283, Locations.tsx:310, Locations.tsx:337
 - Creating: Locations.tsx:231

Select

- Found in [component.ts:26](#) in the backend, and components.tsx:109 in the frontend.

Division

- Found in [project.ts:12](#) in the backend, and projects.tsx:11 in the frontend
- At a high level, it returns all of the projects that are in an “includes” relation with every component

Aggregation with GROUP BY

- Get the total value (price * quantity) of elements per facility
- Backend: [insights.ts:6](#)
- Frontend: home.tsx:38 (fetch) and home.tsx:64 (render)

Aggregation with HAVING

- Find all projects with a cost (price * quantity per component in project) greater than some user-provided threshold
- Backend: [insights.ts:28](#)
- Frontend: home.tsx:53 (fetch) and home.tsx:105 (render)

Nested aggregation with GROUP BY

- Related query code found in:
 - Backend: [shipping.ts:160](#)
 - Frontend: PurchasesTable.tsx:155
- The purpose of the query is to allow a user to see purchases that fit within their specified budget.
- Explanation:
 - The query joins every component with the purchasesincludes relation that includes it, and computes the total price of the purchase.
 - The subquery's HAVING clause is where we filter by the user's budget preference. When grouping by the purchase order number, the nested query then filters for purchases that have a price less than the parsed user value.
 - Following this, a nested subquery counts how many such rows exist in the purchasesincludes relation and includes them in the count if they contain at least 1 component.

AI Tool Use Disclaimer

No AI tools were used for this milestone.